eedi-mining-misconceptions-in-mathematics

2位解法

Rank	2
Team	CQYR
Author	RihanPiggy
Topic ID	551651
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/551651
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

入賞された皆様、おめでとうございます。このような興味深いコンペを開催してくださった主催者の皆様に感謝します。私たちはこの道のりを本当に楽しみ、すべての参加者から多くの刺激を受けました。このように実りあるチームの一員になれたことは幸運であり、チームメンバー全員に感謝したいと思います。ここで私たちのsolutionを共有します。

前処理

NeurIPS 2022で開催された過去のeediコンペから、有用なsubject_metadata.csvを見つけました。このファイルのSubjectIdとSubjectNameは本コンペのものと同一です。subject_metadata.csvには親subjectが含まれているため、このmetadataでvector dbを作成し、train.csvとtest.csvの両方に親subjectの情報を追加しました。

合成データ

合成問題

合成データは3回生成しました。それぞれをgeneration1、generation2、generation3と呼びます。 基本的なapproachは次のとおりです。

- LLMにmisconceptionといくつかの例を与え、問題を生成させる
- qwen-mathで問題を解かせ、正答を得る
- qwen-mathにmisconceptionの制約の下で問題を解かせ、誤答を得る
- gpt-4o-miniで問題の品質を採点させ、scoreが2より大きいもの（最大は5）を選ぶ

各generationの違いは次のとおりです。 Generation1:

- few shot例: train.csvからrandomにsampling

Generation2:

- few shot例: train.csvとGeneration1から、同じmisconceptionを持つ問題をsampling

Generation3:

- Generation3のpromptはこのtech blogのpromptに基づいています
- few shot例: train.csv、Generation1、Generation2からrandomに2問をsampling

misconceptionのaugmentation

misconceptionは短い1文しか含んでいません。misconceptionのembeddingをより意味のあるものにするため、LLMを使って各misconceptionの説明を生成しました。submission時にmisconceptionについて推論を行う必要はないため、このapproachはsubmissionにおいてコストが一切かかりません。 promptは次のとおりです。

retrieverの学習においては、llama3.1-70b-Instructとqwen2.5-72b-Instructによる説明がgpt-4o-miniを上回りました。

```
system_prompt_template =

user_prompt_template =
```

Chain of thought

qwen2.5-32B-Instruct-AWQを使い、後続のretrieveとrerankの追加inputとしてchain-of-thoughtを生成しました。promptは次のとおりです。

```
system_prompt_template =

user_prompt_template =
```

Retrieve

Retrieve modelの学習

retrieverは2種類のpipelineで学習しました。 Pipeline1:

- backbone: Linq-AI-Research/Linq-Embed-Mistral
- loss: Arcface
- 学習と推論でChain of thoughtを追加inputとして使用

Pipeline2:

- backbone: Qwen/Qwen2.5-14B, Qwen/Qwen2.5-32B, Qwen/QwQ-32B-Preview
- loss: MultipleNegativesRankingLoss
- Chain of thoughtなし（推論時間のため）

single modelの性能は次のとおりです。

retriever	合成データ	Private LB	Public LB	推論時間
Linq-AI-Research/Linq-Embed-Mistral	Generation123	0.461	0.484	50 min
Qwen/Qwen2.5-14B	Generation12	0.479	0.507	45 min
Qwen/Qwen2.5-14B	Generation123	0.485	0.492	45 min
Qwen/Qwen2.5-32B	Generation123	0.495	0.536	140 min
Qwen/QwQ-32B-Preview	Generation123	0.500	0.531	140 min

私たちの最良submissionでは、72bのrerankに十分な時間を割けるよう、Mistralと2 x qwen2.5-14Bのensembleを使いました。private LBとpublic LBは0.513、0.530です。

Retriever改善の主要因

1. **大きなmodelを使う** 上の表のとおり、大きいほど良い結果になります。Powerを得るために必要なのはGPUとCredit Cardだけです!!! コンペ期間中は決して請求ページを開かないでください。
    - qwen2.5-14B with Generation12: H100で約2日
    - qwen2.5-14B with Generation123: H100で約3日
    - qwen2.5-32B with Generation123(sampled): H100で約5日
    - QWQ with Generation123(sampled): H100で約5日
2. **合成問題** ほとんどの参加者が合成問題を使ったと思いますが、高品質な問題が多いほど性能は向上します。私たちのチームにとっては、gpt-4o-miniを使って高品質な問題をfilterすることが鍵でした。
3. **misconceptionのaugmentation** misconceptionのaugmentationを使うことで、retrieverの性能が約2-4%と大きく向上しました。
4. **Chain of Thought** CoTも有用です。ただし14Bと32Bのmodelでは、promptにCoTを追加すると推論時間が2倍になります。
5. **Poolingの選択** Qwen modelでは、last token poolingの方がmean poolingより良い性能を達成することが分かりました。

Rerank

retrieveしたcandidateのrankingを洗練させるため、listwise rerankerを使いました。rerankingのプロセスではsliding window approachを採用しました。まず軽量なLLMで8位から17位のcandidateを並べ替え、次により大きなmodelを活用して上位10件のcandidateのrankingを確定させました。

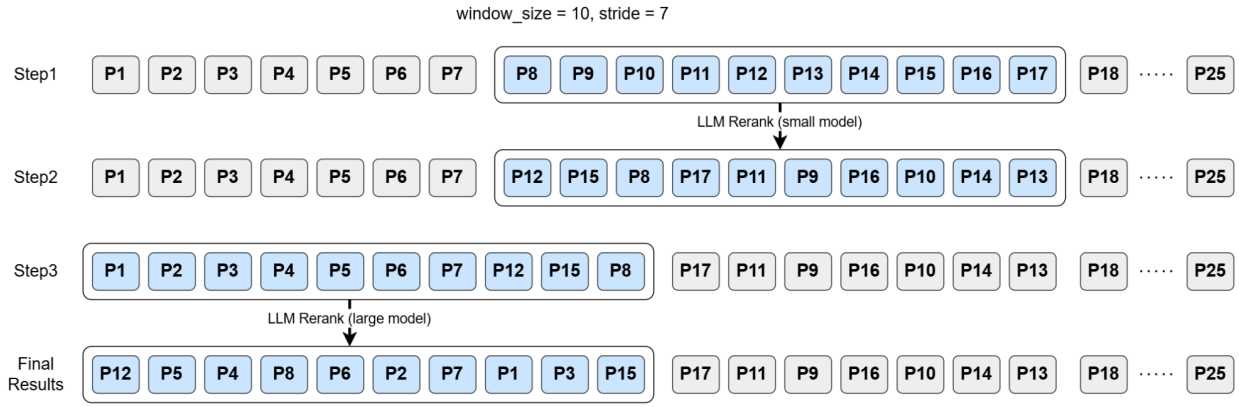
rerankingのLLMは、合成データと学習データを組み合わせてfine-tuningしました。

- **Window 1 (8th ~ 17th)**
    - Qwen2.5-14B-Instruct
- **Window 2 (1st ~ 10th)**
    - Qwen2.5-72B-Instruct
    - Llama-3.3-70B-Instruct

Reranking改善の主要因

1. **大きなmodelを使う** 大きなmodel（例: 72B parameter）は、14Bや32Bのような小さなmodelに比べて一貫して強いvalidation scoreを出すことが分かりました。しかし、これらの大きなmodelは当初Public LBでの成績が悪く、いくらか懸念がありました。それでも私たちはvalidation scoreを信じ、72B modelを最終submissionに含めました（3 submission ruleに特に感謝します!）。最終的に72B modelは際立ったPrivate Leaderboard scoreを出し、入賞の確保に貢献しました。
2. **Chain of Thought** 上記のCoT promptはreranking性能を大きく向上させました。

3. **Sliding Window** rerankingするcandidate数を増やす代わりに、sliding window戦略を複数回適用してtop-10のrankingを洗練させる方が効果的であることが分かりました。



4. **Test-Time Augmentation** 推論時には、一部のmodelでTTAを使いました。逆順のpromptを生成し、そのscoreを通常のpromptのscoreと平均します。この手法で精度がわずかに向上しました。

Rerank modelの学習

LLM rerankerのQLoRA学習codeは、atmacup17の1位solutionに基づいて開発しました。このような優れたsolutionを共有してくださった@kcotton21さんに特に感謝します。

reranking modelの学習で効果的だったtipsをいくつか紹介します。

1. **Listwiseの選択肢をrandom化する** promptに常に上位10件のcandidateを使うのではなく、top-3、top-5、top-15、top-25など、さまざまなtop-Nの組み合わせでpromptを作成しました。
2. **合成データ** retrieverの学習に使った合成データは、rerankerの学習にも役立ちました。合計で、学習データ8,000 record（2 epoch分）と合成データ14,000 recordで学習し、合わせて22,000 recordのdatasetになりました。
3. **Negative Sample Mining** negative sampleのminingには、fine-tuningした`dunzhang/stella_en_400M_v5` modelとTF-IDFを組み合わせたhybrid retrieverを使いました。22,000件のpositive sampleそれぞれについて、この構成で対応するnegative sampleをminingしました。

学習時間

Qwen2.5-14B: H100で約2時間 Qwen2.5-72B: H100で約8時間

量子化

LLM rerankerの量子化にはintel/auto-round libraryを使いました。AutoGPTQやAutoAWQと比べて、このlibraryは使いやすく、精度低下も最小限（通常2%未満）でした。さらに、vLLMと互換性のあるmodelを生成できました。

qwen2.5-72b-Instructは、intermediate_size(29568)のためにmulti GPUで動かす際にいくつか問題があります。gptqのdocumentで提供されている回避策に従い、weightを29696にpaddingしてから量子化を行いました。

calibrationには学習datasetを使いました。量子化のparameterは以下のとおりです。

```
bits, group_size, sym = 4, 128,
autoround = AutoRound(
    model, tokenizer, =bits, =group_size, =sym, =calib_prompts, =256,
```

```
    =512,
    =500,
)
```

推論

推論にはvLLMを使っています。

`enabling_prefix_cache`を`True`に設定することで、推論時間を約10%削減できました。

jagatkiranさんが72B LLM modelでの推論に関する知見を共有してくれました。このコンペでは大きなmodelほど良い性能を出す傾向があり、これは私たちにとって非常に役立ちました。

rerankerは、特定のtokenに+100のweightを割り当てる形で`logits_processors`と`logprobs`を使って実装しました。このapproachにより、rankerの枠組みを確立できました。classification headを直接使うことも試しましたが、結果は満足のいくものではなく、vLLMでの推論も容易ではありませんでした。この方法は今後のコンペにおけるparadigmになり得ると考えています。

Ablation

Baseline (retriever)	Qwen14B	Qwen72B	Llama70B	8-17th Qwen14B	Private LB	Public LB
✅					0.513	0.530
✅	✅				0.568	0.583
✅		✅			0.593	0.582
✅	✅	✅	✅		0.596	0.609
✅	✅	✅	✅	✅	**0.604**	0.622

最終日には、Llama70Bの代わりにNexusflow/Athene-V2-Chatのfine-tuningを試しました。残念ながら、このmodelを使ったsubmissionはgpuとtimeoutの問題でtimeoutになりましたが、leaderboardでは0.609という非常に印象的な性能を示しました。

train code: https://github.com/wangqihanginthesky/Eedi_kaggle\ inference code: https://www.kaggle.com/code/honglihang/2nd-place-inference-code

Reference

LLMにおける合成データセットによる数学推論タスクの精度向上の検討 NeurIPS 2022 CausalML Challenge: Causal Insights for Learning Paths in Education A Setwise Approach for Effective and Highly Efficient Zero-shot Ranking with Large Language Models Qwen2.5-72B & Llama 3.3-70B on 2xT4(questionable performance) Optimize Weight Rounding via Signed Gradient Descent for the Quantization of LLMs 1st place solution from atmacup17 Winning Amazon KDD Cup'24 Qwen GPTQ Troubleshooting

補足Q&A

Yi-Chia Chen — 質問

> 「NeurIPS 2022で開催された過去のeediコンペから、有用なsubject_metadata.csvを見つけました。このファイルのSubjectIdとSubjectNameは本コンペのものと同一です。subject_metadata.csvには親subjectが含まれているため、このmetadataでvector dbを作成し、train.csvとtest.csvの両方に親subjectの情報を追加しました。」

素晴らしいですね。親subjectの情報がmodelの性能をどれくらい改善したか分析しましたか。

kuto — 質問

おめでとうございます。solutionを共有していただきありがとうございます。とても興味深く読みました。

2つ質問があります。

1. rerankerの学習時と推論時で、promptと出力の形式は異なりますか。学習時は1つのmisconceptionに対してYes/Noを予測し、推論時は複数のmisconceptionを並べ替える、と理解しました。これで合っていますか。
2. special tokenに100のweightを割り当てる部分がよく理解できませんでした。もう少し詳しく説明していただけますか。

> rerankerは、特定のtokenに+100のweightを割り当てる形でlogits_processorsとlogprobsを使って実装しました。

ktr — 質問

賞金圏の金メダル獲得おめでとうございます。私の過去のsolutionがお役に立てたと聞いて嬉しいです。

投稿を拝見すると、72bをうまく活用されたことが分かります。 72bはQLoRAで学習し、Adapterをmergeしてから量子化したのでしょうか。経験的には、QLoRAで学習したAdapterをmergeすると性能が大きく低下すると思います。

Qihang Wang — 回答

ご質問ありがとうございます。

学習と推論のプロセスは一貫しており、どちらもA, B, C, D …のような選択肢を扱います。

special tokenのweightについて: これらの選択肢（A, B, C, D,…）に対して+100のlogit_biasを適用し、modelがこれらの文字を優先して出力するようにしています。（logitprocessorの**call**で `scores[self.allowed_ids] += 100` を設定）

RihanPiggy — 回答

atmacupでのsolutionを本当にありがとうございました。そしておめでとうございます。 はい、72bはQLoRaで学習し、Adapterをmergeしてから量子化を行いました。私たちのoffline実験では、性能はそれほど低下しませんでした（0.7011 -> 0.6988）。 rerankの学習は@yokuyamaさんが開発したもので、量子化の際にhyper-parameter

を最適化したと聞いています。auto-roundは非常に強力で使いやすく、私たちの実験ではauto-awqより優れていました。

RihanPiggy — 回答

コンペ開始時に外部データを集めたので、十分には確かではありません。個人的には、14Bと32Bのretrieverがこの情報を使っていないため、効果はわずかだと思います。ただし、親subjectはChain-of-thought、Rerank、合成データ生成のinputとして使っており、modelに追加のcontextを与えている可能性があります。